

```

/* =====
  Fluent - English AI Tutor
  app.js - all client logic. Talks to the backend only through api.js.
  ===== */

const STORAGE_KEY = "fluent_state_v1";

/* ----- Modes ----- */
const MODES = {
  free: {
    title: "Free Conversation",
    desc: "Talk about anything, at your own pace",
    emoji: "💬",
    brief: "General conversation partner. Discuss daily life, study, work, econom
  },
  ielts: {
    title: "IELTS Speaking",
    desc: "A full mock speaking test with a band estimate at the end",
    emoji: "🎓",
    brief: "Run a realistic IELTS Speaking mock test. Move through Part 1 (short
  },
  debate: {
    title: "Debate",
    desc: "Pick a side and defend your point of view",
    emoji: "⚖️",
    brief: "Choose (or let the user choose) a debatable topic and argue a positio
  },
  news: {
    title: "News Discussion",
    desc: "Talk through current events (uses web search when available)",
    emoji: "📰",
    brief: "Discuss current events and news topics. Never invent a headline, stat
  },
  interview: {
    title: "Job Interview",
    desc: "Practice answering (and asking) interview questions in English",
    emoji: "💼",
    brief: "Act as a hiring manager conducting a realistic job interview in Engli
  },
  daily: {
    title: "Daily English",
    desc: "Practical, everyday situations and small talk",
    emoji: "☕",
    brief: "Practice everyday, practical English: ordering food, small talk, appo

```

```

    },
    vocab: {
      title: "Vocabulary Practice",
      desc: "Build and review vocabulary suited to your level",
      emoji: "📖",
      brief: "Focus the conversation on vocabulary building. When the user uses a v
    },
    grammar: {
      title: "Grammar Practice",
      desc: "Targeted explanations and short exercises",
      emoji: "🧩",
      brief: "Focus on grammar. Explain rules simply (in Arabic if the user asks fo
    },
  };

const MODE_ORDER = ["free", "ielts", "debate", "news", "interview", "daily", "voc

/* ----- State ----- */
function loadState() {
  const defaults = {
    theme: "light",
    level: "auto",
    correctionMode: "immediate",
    currentMode: "free",
    sessions: 0,
    sessionStarted: false,
    vocab: [], // { word, note, addedAt }
    mistakes: {}, // { "Grammar:subject-verb agreement": { count, type, ex
    ieltsScores: [], // { date, overall, breakdown, tips }
  };
  try {
    const raw = localStorage.getItem(STORAGE_KEY);
    if (!raw) return defaults;
    return Object.assign(defaults, JSON.parse(raw));
  } catch (e) {
    return defaults;
  }
}

function saveState() {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(state));
}

let state = loadState();
let conversation = []; // { role: 'user'|'assistant', content: string } - curr
let ieltsPart = 1;
let lastSpokenText = "";

```

```

let recognition = null;
let recognizing = false;

/* ----- DOM ----- */
const $ = (id) => document.getElementById(id);
const chatLog = $("chatLog");
const chatScroll = $("chatScroll");
const composerForm = $("composerForm");
const composerInput = $("composerInput");
const micBtn = $("micBtn");
const micStatus = $("micStatus");
const replayBtn = $("replayBtn");
const rateSlider = $("rateSlider");
const sidebar = $("sidebar");
const panel = $("panel");
const sidebarToggle = $("sidebarToggle");
const panelToggle = $("panelToggle");
const modeList = $("modeList");
const levelSelect = $("levelSelect");
const correctionToggle = $("correctionModeToggle");
const themeToggle = $("themeToggle");
const themeLabel = $("themeLabel");
const resetDataBtn = $("resetDataBtn");
const currentModeTitle = $("currentModeTitle");
const currentModeDesc = $("currentModeDesc");
const ieltsBar = $("ieltsBar");
const ieltsPartLabel = $("ieltsPartLabel");
const endIeltsBtn = $("endIeltsBtn");
const toast = $("toast");
const statSessions = $("statSessions");
const statVocab = $("statVocab");
const statMistakes = $("statMistakes");

/* ===== Rendering ===== */

function renderModes() {
  modeList.innerHTML = "";
  MODE_ORDER.forEach((key) => {
    const m = MODES[key];
    const btn = document.createElement("button");
    btn.className = "mode-item" + (key === state.currentMode ? " active" : "");
    btn.innerHTML = `${m.emoji}<span>${m.title}</spa
    btn.addEventListener("click", () => switchMode(key));
    modeList.appendChild(btn);
  });
}

```

```

function renderTopbar() {
  const m = MODES[state.currentMode];
  currentModeTitle.textContent = m.title;
  currentModeDesc.textContent = m.desc;
  ieltsBar.hidden = state.currentMode !== "ielts";
  if (state.currentMode === "ielts") ieltsPartLabel.textContent = `Part ${ieltsPa
}

function renderStats() {
  statSessions.textContent = state.sessions;
  statVocab.textContent = state.vocab.length;
  statMistakes.textContent = Object.keys(state.mistakes).length;
}

function renderVocabPanel() {
  const list = $("vocabList");
  if (!state.vocab.length) {
    list.innerHTML = `<div class="empty-hint">No words saved yet. Look for "Add t
    return;
  }
  list.innerHTML = state.vocab
    .slice()
    .reverse()
    .map(
      (v) => `<div class="card"><div class="card-title">${escapeHtml(v.word)}</di
    )
    .join("");
}

function renderMistakesPanel() {
  const list = $("mistakesList");
  const entries = Object.entries(state.mistakes).sort((a, b) => b[1].count - a[1]
  if (!entries.length) {
    list.innerHTML = `<div class="empty-hint">Nothing tracked yet – keep chatting
    return;
  }
  list.innerHTML = entries
    .map(([key, v]) => {
      const example = v.examples[v.examples.length - 1] || {};
      return `<div class="card">
        <span class="count-badge">${v.count}x</span>
        <div class="card-title">${escapeHtml(v.type)}</div>
        ${example.original ? `<div class="card-sub">"${escapeHtml(example.origina
        </div>`;
      })
    }
    .join("");
}

```

```

function renderReportPanel(report) {
  const empty = $("reportEmpty");
  const content = $("reportContent");
  if (!report) {
    empty.style.display = "block";
    content.innerHTML = "";
    return;
  }
  empty.style.display = "none";
  const b = report.breakdown || {};
  content.innerHTML = `
    <div class="band-overall">
      <div class="b-big">${escapeHtml(String(report.overall ?? "-"))}</div>
      <div class="b-note">Estimated overall band – not an official score</div>
    </div>
    <div class="band-grid">
      <div class="band-cell"><div class="b-score">${escapeHtml(String(b.fluency ?
      <div class="band-cell"><div class="b-score">${escapeHtml(String(b.lexical ?
      <div class="band-cell"><div class="b-score">${escapeHtml(String(b.grammar ?
      <div class="band-cell"><div class="b-score">${escapeHtml(String(b.pronuncia
    </div>
    <div class="card"><div class="card-title">Why this estimate</div><div class="
    <div class="card" style="margin-top:8px"><div class="card-title">How to move
  `;
}

function escapeHtml(str) {
  return String(str)
    .replace(/&/g, "&amp;")
    .replace(/</g, "&lt;")
    .replace(/>/g, "&gt;");
}

function scrollToBottom() {
  chatScroll.scrollTop = chatScroll.scrollHeight;
}

function showToast(msg) {
  toast.textContent = msg;
  toast.classList.add("show");
  setTimeout(() => toast.classList.remove("show"), 2200);
}

/* ----- Chat bubbles ----- */

function appendMessage(role, text) {

```

```

    if (!text) return;
    const wrap = document.createElement("div");
    wrap.className = `msg ${role === "user" ? "user" : "bot"}`;
    const bubble = document.createElement("div");
    bubble.className = "bubble";
    bubble.textContent = text;
    wrap.appendChild(bubble);
    chatLog.appendChild(wrap);
    scrollToBottom();
}

function appendSystemNote(text) {
    const el = document.createElement("div");
    el.className = "system-note";
    el.textContent = text;
    chatLog.appendChild(el);
    scrollToBottom();
}

function appendCorrectionCard(c) {
    const el = document.createElement("div");
    el.className = "correction-card";
    el.innerHTML = `
        <span class="c-tag">${escapeHtml(c.type || "Correction")}</span>
        <div class="c-orig">${escapeHtml(c.original || "")}</div>
        <div class="c-fix">→ ${escapeHtml(c.fix || "")}</div>
        ${c.explanation ? `<div class="c-why">${escapeHtml(c.explanation)}</div>` : ""}
    `;
    chatLog.appendChild(el);
    scrollToBottom();

    if (c.type && c.original) {
        const key = c.type + ":" + c.original.slice(0, 40);
        const bucket = state.mistakes[c.type] || { count: 0, type: c.type, examples:
        bucket.count += 1;
        bucket.examples.push({ original: c.original, fix: c.fix });
        if (bucket.examples.length > 5) bucket.examples.shift();
        state.mistakes[c.type] = bucket;
        saveState();
        renderMistakesPanel();
        renderStats();
    }
}

function appendVocabSuggest(v) {
    const el = document.createElement("div");
    el.className = "vocab-suggest";

```

```

const btnId = "vocabbtn-" + Math.random().toString(36).slice(2, 8);
el.innerHTML = `<span><strong>${escapeHtml(v.word)}</strong> - ${escapeHtml(v.n
chatLog.appendChild(el);
scrollToBottom();
$(btnId).addEventListener("click", () => {
  addVocab(v.word, v.note);
  $(btnId).textContent = "Added ✓";
  $(btnId).disabled = true;
});
}

function addVocab(word, note) {
  if (state.vocab.some((v) => v.word.toLowerCase() === word.toLowerCase())) {
    showToast("Already in your vocabulary list");
    return;
  }
  state.vocab.push({ word, note, addedAt: Date.now() });
  saveState();
  renderVocabPanel();
  renderStats();
  showToast(`Added "${word}" to vocabulary`);
}

let typingEl = null;
function showTyping() {
  typingEl = document.createElement("div");
  typingEl.className = "msg bot";
  typingEl.innerHTML = `<div class="bubble typing-dots"><span></span><span></span></div>`;
  chatLog.appendChild(typingEl);
  scrollToBottom();
}

function hideTyping() {
  if (typingEl) {
    typingEl.remove();
    typingEl = null;
  }
}

/* ===== Reading-aloud practice ===== */
/* The tutor can hand back a short passage (any topic, requested in plain
English) via the [[READING_PASSAGE]] tag. The user reads it aloud, we
transcribe with Web Speech API, do a lightweight word-match against the
original text (no real phonetic/accent analysis is possible in-browser),
highlight likely trouble spots, and send the transcript + mismatches back
to the model for qualitative accent/fluency feedback. */

let readingCounter = 0;

```

```

let activeReading = null; // { id, recognition }

function appendReadingCard(passage) {
  if (!passage || !passage.text) return;
  readingCounter += 1;
  const id = "reading-" + readingCounter;
  const words = passage.text.split(/\s+/);

  const wrap = document.createElement("div");
  wrap.className = "reading-card";
  wrap.id = id;
  wrap.innerHTML = `
    <div class="reading-topic">📖 ${escapeHtml(passage.topic || "Reading passage")}
    <div class="reading-passages" id="${id}-text">${words
      .map((w, i) => `<span class="word" data-i="${i}">${escapeHtml(w)}</span>`)
      .join(" ")}</div>
    <div class="reading-controls">
      <button type="button" class="pill-btn" id="${id}-start">🎤 Start reading</button>
      <button type="button" class="pill-btn" id="${id}-stop" hidden>🔴 Stop &
      <span class="reading-status" id="${id}-status"></span>
    </div>
    <div class="reading-caveat">Feedback is based on what speech recognition tran
  `;
  chatLog.appendChild(wrap);
  scrollToBottom();

  const startBtn = $(id + "-start");
  const stopBtn = $(id + "-stop");
  const statusEl = $(id + "-status");

  startBtn.addEventListener("click", () => startReading(passage, words, id, start
  stopBtn.addEventListener("click", () => {
    if (activeReading && activeReading.id === id) activeReading.recognition.stop(
  });
}

function startReading(passage, words, id, startBtn, stopBtn, statusEl) {
  const SR = window.SpeechRecognition || window.webkitSpeechRecognition;
  if (!SR) {
    statusEl.textContent = "Voice input isn't supported in this browser.";
    return;
  }
  if (activeReading) activeReading.recognition.stop();

  const rec = new SR();
  rec.lang = "en-US";
  rec.continuous = true;

```



```

rec.interimResults = true;
let finalTranscript = "";
activeReading = { id, recognition: rec };

rec.onstart = () => {
  startBtn.hidden = true;
  stopBtn.hidden = false;
  statusEl.textContent = "

```

```

    for (let j = sPointer; j < Math.min(spokenWords.length, sPointer + WINDOW + 1)
        if (spokenWords[j] === ow) {
            sPointer = j + 1;
            return true;
        }
    }
    unmatched.push(ow);
    return false;
});

const container = $(id + "-text");
container.querySelectorAll(".word").forEach((span, i) => {
    span.classList.add(matchFlags[i] ? "matched" : "unmatched");
});

const meaningful = origWords.filter(Boolean);
const matchedCount = origWords.reduce((acc, ow, i) => acc + (ow && matchFlags[i]
const pct = meaningful.length ? Math.round((matchedCount / meaningful.length) *

if (!transcript) {
    statusEl.textContent = "No speech was captured – try again a bit closer to th
    return;
}

statusEl.textContent = `Recognized match: ${pct}% of words – getting feedback...`
requestReadingFeedback(passage, transcript, unmatched, pct);
}

async function requestReadingFeedback(passage, transcript, unmatchedWords, pct) {
    const content =
        `I just read this passage aloud:\n"${passage.text}"\n\n` +
        `Speech recognition transcribed my reading as:\n"${transcript}"\n\n` +
        `Words it didn't match cleanly (${unmatchedWords.length} of ${passage.text.sp
            unmatchedWords.join(", ") || "none"
        }.\nOverall word-match: ${pct}%.\n\n` +
        `Please give me feedback focused on my accent, pronunciation, and reading flu

appendSystemNote(`📖 Reading attempt submitted (${pct}% words matched) – gettin
conversation.push({ role: "user", content });
showTyping();
sendBtnDisabled(true);
try {
    const reply = await FluentAPI.chat(conversation, buildSystemPrompt());
    hideTyping();
    const parsed = processAssistantReply(reply);
    if (parsed.text) {
        appendMessage("bot", parsed.text);
    }
}

```

```

        conversation.push({ role: "assistant", content: reply });
        speak(parsed.text);
    }
    parsed.corrections.forEach(appendCorrectionCard);
    parsed.vocabSuggestions.forEach(appendVocabSuggest);
    parsed.readingPassages.forEach(appendReadingCard);
} catch (err) {
    hideTyping();
    appendSystemNote("⚠️ " + err.message);
} finally {
    sendBtnDisabled(false);
}
}

$("readingBtn").addEventListener("click", () => {
    const topic = prompt("What should the reading passage be about? (e.g. history,
    if (topic && topic.trim()) {
        sendMessage(`Can you give me a short passage to read aloud about ${topic.
    }
});

/* ===== Structured tag parsing ===== */
/* The model is asked (via the system prompt) to wrap structured data in
   simple [[TAG]]...json...[/TAG]] markers. We strip them from the visible
   text and parse them into UI elements. Kept deliberately simple (no real
   XML/JSON schema validation) since this is a teaching demo, not a
   production parser. */

function extractTag(text, tag) {
    const re = new RegExp(`\\[\\[${tag}\\]\\]\\s*([\\s\\S]*?)\\[\\[\\/\\${tag}\\]\\]\\s*`, "g");
    const found = [];
    let m;
    let stripped = text;
    while ((m = re.exec(text)) !== null) {
        found.push(m[1].trim());
    }
    stripped = stripped.replace(re, "").trim();
    return { found, stripped };
}

function tryParseJSON(str) {
    try {
        return JSON.parse(str);
    } catch (e) {
        return null;
    }
}

```

```

function processAssistantReply(raw) {
  let text = raw;

  // IELTS part marker: [[IELTS_PART]]2[[/IELTS_PART]]
  let r = extractTag(text, "IELTS_PART");
  text = r.stripped;
  if (r.found.length) {
    const n = parseInt(r.found[r.found.length - 1], 10);
    if (!isNaN(n)) {
      ieltsPart = n;
      ieltsPartLabel.textContent = `Part ${n}`;
    }
  }
}

// Correction block(s): [[CORRECTION]]{...}[[/CORRECTION]]
r = extractTag(text, "CORRECTION");
text = r.stripped;
const corrections = r.found.map(tryParseJSON).filter(Boolean);

// Vocabulary suggestion(s): [[VOCAB]]{...}[[/VOCAB]]
r = extractTag(text, "VOCAB");
text = r.stripped;
const vocabSuggestions = r.found.map(tryParseJSON).filter(Boolean);

// IELTS / session report: [[IELTS_REPORT]]{...}[[/IELTS_REPORT]]
r = extractTag(text, "IELTS_REPORT");
text = r.stripped;
const report = r.found.map(tryParseJSON).filter(Boolean)[0] || null;

// Level estimate (auto mode): [[LEVEL_ESTIMATE]]B1[[/LEVEL_ESTIMATE]]
r = extractTag(text, "LEVEL_ESTIMATE");
text = r.stripped;
const levelEstimate = r.found[r.found.length - 1] || null;

// Reading-aloud passage: [[READING_PASSAGE]]{...}[[/READING_PASSAGE]]
r = extractTag(text, "READING_PASSAGE");
text = r.stripped;
const readingPassages = r.found.map(tryParseJSON).filter(Boolean);

return { text: text.trim(), corrections, vocabSuggestions, report, levelEstimate }
}

/* ===== System prompt ===== */

function buildSystemPrompt() {
  const mode = MODES[state.currentMode];

```

```

    const level = state.level === "auto" ? "auto-detect the level from the user's E

    return `You are "Fluent", a warm, encouraging, highly qualified English tutor.

CURRENT MODE: ${mode.title}. ${mode.brief}

LEARNER LEVEL: ${level}. Adjust vocabulary, sentence complexity, and pace to this

CORE RULES:
- Sound like a real person, not a scripted bot with memorized questions. Follow t
- Ask natural follow-up questions instead of ending the conversation quickly, esp
- Remember and use context from earlier in this session.
- Don't interrupt or redirect a user who is in the middle of a longer answer (e.g
- Don't correct everything – that's annoying. Focus on errors that are real mista
- Never invent news, statistics, or claims about recent/current events. If nothin
- Encourage the user to keep using English rather than translating everything for
- If assessing IELTS speaking, use the official public IELTS band descriptors as

CORRECTION MODE: ${state.correctionMode === "immediate" ? "Correct mistakes as th
${state.correctionMode === "immediate" ? `When the user's last message contains a
[[CORRECTION]]{"original":"<exact original phrase>","fix":"<corrected phrase>","t

VOCABULARY COACHING: when the user uses a very basic word that could be upgraded
[[VOCAB]]{"word":"<the stronger word or phrase>","note":"<short note on nuance /
Don't force this on every message – only when it's genuinely useful, and never tu

IELTS SCORING: only when the user asks to finish/end an IELTS speaking session, r
[[IELTS_REPORT]]{"overall":6.5,"breakdown":{"fluency":6,"lexical":7,"grammar":6,"

READING-ALoud PRACTICE (available in every mode): whenever the user asks you for
[[READING_PASSAGE]]{"topic":"<short topic label>","text":"<3-6 natural sentences,
The app records the user reading this aloud and sends you back what speech recogn
- Be explicit that this is inferred only from word-level differences between the
- Where the pattern plausibly fits, you may mention common pronunciation patterns
- Comment on pace/fluency if the transcript came back much shorter or longer than
- End with one or two concrete things to practice next.

Never show these bracket tags' raw contents to the user as prose – they are parse
}

/* ===== Sending flow ===== */

async function sendUserMessage(text) {
    text = text.trim();
    if (!text) return;

    if (!state.sessionStarted) {

```

```

    state.sessionStarted = true;
    state.sessions += 1;
    saveState();
    renderStats();
}

appendMessage("user", text);
conversation.push({ role: "user", content: text });
composerInput.value = "";
autoGrowInput();

showTyping();
sendBtnDisabled(true);

try {
    let searchContext = "";
    if (state.currentMode === "news") {
        const s = await FluentAPI.search(text);
        if (s.available && s.results && s.results.length) {
            searchContext =
                "\n\nSEARCH_RESULTS (for this turn only, use if relevant, cite the sour
                s.results.map((r, i) => `${i + 1}. ${r.title} - ${r.snippet} (${r.url})
        } else {
            searchContext = "\n\nSEARCH_RESULTS: none available this turn - do not cl
        }
    }

    const system = buildSystemPrompt() + searchContext;
    const reply = await FluentAPI.chat(conversation, system);
    hideTyping();

    const parsed = processAssistantReply(reply);
    if (parsed.text) {
        appendMessage("bot", parsed.text);
        conversation.push({ role: "assistant", content: reply });
        speak(parsed.text);
    }
    parsed.corrections.forEach(appendCorrectionCard);
    parsed.vocabSuggestions.forEach(appendVocabSuggest);
    parsed.readingPassages.forEach(appendReadingCard);
    if (parsed.report) {
        state.ieltsScores.push({ date: Date.now(), ...parsed.report });
        saveState();
        renderReportPanel(parsed.report);
        openPanelTab("report");
    }
    if (parsed.levelEstimate) {

```

```

        currentModeDesc.textContent = `${MODES[state.currentMode].desc} · est. leve
    }
} catch (err) {
    hideTyping();
    appendSystemNote("⚠️ " + err.message);
} finally {
    sendBtnDisabled(false);
}
}

function sendBtnDisabled(disabled) {
    $("sendBtn").disabled = disabled;
    micBtn.disabled = disabled;
}

/* ===== Mode switch ===== */

function switchMode(key) {
    state.currentMode = key;
    state.sessionStarted = false;
    conversation = [];
    ieltsPart = 1;
    saveState();
    renderModes();
    renderTopbar();
    chatLog.innerHTML = "";
    renderReportPanel(null);
    appendSystemNote(`Switched to ${MODES[key].title}. ${MODES[key].desc}.`);
    if (key === "ielts") {
        appendSystemNote("Say \"I'm ready\" (or anything) to begin the mock test. Use \"
    }
    if (window.innerWidth <= 760) sidebar.classList.remove("open");
}

/* ===== IELTS end ===== */

endIeltsBtn.addEventListener("click", () => {
    sendUserMessage("Please end the IELTS speaking session now and give me my band
});

/* ===== Speech (input) ===== */

function setupSpeechRecognition() {
    const SR = window.SpeechRecognition || window.webkitSpeechRecognition;
    if (!SR) {
        micBtn.disabled = true;
        micBtn.title = "Voice input is not supported in this browser";
    }
}

```

```

        return;
    }
    recognition = new SR();
    recognition.lang = "en-US";
    recognition.interimResults = true;
    recognition.continuous = false;

    recognition.onstart = () => {
        recognizing = true;
        micBtn.classList.add("recording");
        micStatus.textContent = "🎧 Listening... speak in English";
    };
    recognition.onerror = (e) => {
        micStatus.textContent = e.error === "not-allowed" ? "Microphone access was bl
    };
    recognition.onend = () => {
        recognizing = false;
        micBtn.classList.remove("recording");
        setTimeout(() => (micStatus.textContent = ""), 1500);
    };
    recognition.onresult = (event) => {
        let transcript = "";
        for (let i = 0; i < event.results.length; i++) transcript += event.results[i]
        composerInput.value = transcript;
        autoGrowInput();
    };
}

micBtn.addEventListener("click", () => {
    if (!recognition) return;
    if (recognizing) {
        recognition.stop();
    } else {
        try {
            recognition.start();
        } catch (e) {
            /* already started */
        }
    }
});

/* ===== Speech (output) ===== */

function speak(text) {
    lastSpokenText = text;
    if (!("speechSynthesis" in window)) return;
    window.speechSynthesis.cancel();

```



```

    const utter = new SpeechSynthesisUtterance(text);
    utter.lang = "en-US";
    utter.rate = parseFloat(rateSlider.value) || 0.9;
    window.speechSynthesis.speak(utter);
  }

  replayBtn.addEventListener("click", () => {
    if (!lastSpokenText) {
      showToast("Nothing to replay yet");
      return;
    }
    speak(lastSpokenText);
  });

  /* ===== Composer ===== */

  composerForm.addEventListener("submit", (e) => {
    e.preventDefault();
    sendUserMessage(composerInput.value);
  });

  composerInput.addEventListener("keydown", (e) => {
    if (e.key === "Enter" && !e.shiftKey) {
      e.preventDefault();
      sendUserMessage(composerInput.value);
    }
  });

  composerInput.addEventListener("input", autoGrowInput);
  function autoGrowInput() {
    composerInput.style.height = "auto";
    composerInput.style.height = Math.min(composerInput.scrollHeight, 120) + "px";
  }

  /* ===== Panel / sidebar UI ===== */

  function openPanelTab(tab) {
    document.querySelectorAll(".panel-tab").forEach((t) => t.classList.toggle("active"));
    document.querySelectorAll(".panel-view").forEach((v) => v.classList.toggle("active"));
    if (window.innerWidth <= 1080) panel.classList.add("open");
  }

  document.querySelectorAll(".panel-tab").forEach((btn) => {
    btn.addEventListener("click", () => openPanelTab(btn.dataset.tab));
  });

  sidebarToggle.addEventListener("click", () => sidebar.classList.toggle("open"));

```

```

panelToggle.addEventListener("click", () => panel.classList.toggle("open"));

correctionToggle.querySelectorAll(".toggle-btn").forEach((btn) => {
  btn.addEventListener("click", () => {
    correctionToggle.querySelectorAll(".toggle-btn").forEach((b) => b.classList.r
    btn.classList.add("active");
    state.correctionMode = btn.dataset.mode;
    saveState();
  });
});

levelSelect.addEventListener("change", () => {
  state.level = levelSelect.value;
  saveState();
});

themeToggle.addEventListener("click", () => {
  state.theme = state.theme === "light" ? "dark" : "light";
  applyTheme();
  saveState();
});

function applyTheme() {
  document.documentElement.setAttribute("data-theme", state.theme);
  themeLabel.textContent = state.theme === "light" ? "Dark mode" : "Light mode";
}

resetDataBtn.addEventListener("click", () => {
  if (confirm("This clears all locally saved vocabulary, mistake tracking, and IE
  localStorage.removeItem(STORAGE_KEY);
  location.reload();
})
});

/* ===== Init ===== */

function init() {
  applyTheme();
  levelSelect.value = state.level;
  correctionToggle.querySelectorAll(".toggle-btn").forEach((b) => b.classList.tog
  renderModes();
  renderTopbar();
  renderStats();
  renderVocabPanel();
  renderMistakesPanel();
  renderReportPanel(state.ieltsScores.length ? state.ieltsScores[state.ieltsScore
  setupSpeechRecognition();

```

```
    appendSystemNote(`Welcome back. You're in ${MODES[state.currentMode].title} mod  
  }
```

```
init();
```